

# CSE464Project3

## Features

### Part 1

- Parse a directed graph from a DOT file
- Output graph contents to a text file
- Add a single node
- Add multiple nodes
- Add an edge

### Part 2

- Remove a single node
- Remove multiple nodes
- Remove an edge

### Part 3

- Refactor the graph search implementation
- Apply the Template Pattern to reduce duplicated search logic
- Apply the Strategy Pattern to select a search algorithm at runtime
- Add Random Walk search

## Build

Compile the project:

```
mvn clean compile
```

Run tests:

```
mvn test
```

## Running the Application

Run the app with an input DOT file and an output directory:

```
mvn exec:java -Dexec.mainClass="edu.asu.cse464.dot.App" -Dexec.args="input.dot  
output"
```

### Arguments

- input.dot = path to the DOT input file
- output = directory where generated files will be written

## Example Input

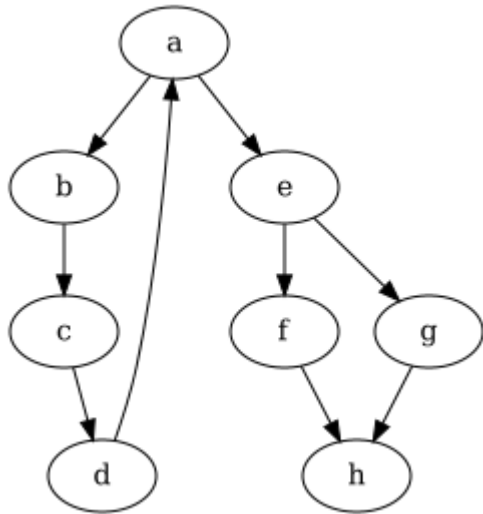
```
digraph {  
    a -> b;  
    b -> c;  
    c -> d;
```

```

    d -> a;
    a -> e;
    e -> f;
    e -> g;
    f -> h;
    g -> h;
}

```

## Example output



The demo searches from node a to node h.

Example console output:

```

BFS a -> h: a -> e -> f -> h
DFS a -> h: a -> e -> g -> h
Random walk a -> h run 1: a -> e -> f -> h
Random walk a -> h run 2: a -> e -> g -> h
Random walk a -> h run 3: a -> e -> f -> h

```

Random walk may produce different valid paths on different runs.

## Output Files

The application generates:

- graph.txt — text representation of the graph
- graph.dot — DOT output
- graph.png — graph visualization image

## Running Specific Tests

Run all tests:

```
mvn test
```

Run one test class:

```
mvn -Dtest=DotGraphIOTest test
```

Run one specific test method:

```
mvn -Dtest=DotGraphIOTest#graph_search_bfs_finds_a_to_h_path test
```

## Search Algorithms

### BFS

Breadth-First Search explores level by level and finds the shortest path in terms of edge count.

### DFS

Depth-First Search explores one path deeply before backtracking.

### Random Walk

Random Walk explores using randomized next choices and may return different valid paths across runs.

## Example API Usage

```
DirectedGraph graph = DotGraphIO.parseGraph("input.dot");

Node src = graph.getNode("a");
Node dst = graph.getNode("h");

Path bfsPath = graph.GraphSearch(src, dst, Algorithm.BFS);
Path dfsPath = graph.GraphSearch(src, dst, Algorithm.DFS);
Path randomPath = graph.GraphSearch(src, dst, Algorithm.RANDOM_WALK);

System.out.println(bfsPath);
System.out.println(dfsPath);
System.out.println(randomPath);
```